

Regis Michael de Sousa Bastos - 85981913436

1. a

É a classe central. Todo sistema vai ser voltado a essa classe.

1. b

O modificador private previne que a variavel seja chamada diretamente por outras classes. Para acessar precisa usar o metodo da classe get que só é disponibilizado depois de criar uma instancia da classe.

1. c

Método get

1. d

```
public Tarefa(String titulo, String descricao , boolean concluido) {  
    this.titulo = titulo;  
    this.descricao = descricao;  
    this.concluido = concluido;  
}
```

2.

getter e setter titulo:

```
public String getTitulo() {  
    return titulo;  
}
```

```
public void setTitulo(String titulo) {  
    this.titulo = titulo;  
}
```

getter e setter descricao:

```
public String getDescricao() {  
    return descricao;  
}
```

```
public void setDescricao(String descricao) {  
    this.descricao = descricao;  
}
```

getter e setter concluida:

```
public boolean isConcluidoa() {  
    return concluida;  
}
```

```
public void setConcluida(boolean concluida) {  
    this.concluida = concluida;  
}
```

Encapsulamento. Para acessar o metodo da classe, você precisa criar um new Classe(). Com isso reduz drasticamente a alteração de valores de variavel de forma errada. Dentre outras vantagens.

3.

Model: Tarefa, TarefaPrioritaria
Controller: TaskManager
App: Main
Exceptions: TarefaNaoEncontradaException

Model – Pasta utilizada para todos os modelos de objetos do mundo real.
Controller – Efetua o controle dos dados que é recebido e do que é enviado.
App – Pasta principal, usada para iniciar um software
Exceptions – Todas exceptions esperadas que o desenvolvedor deseja tratar.

4. a
Herança

4. b
Porque TarefaPrioritaria possui os mesmos atributos de Tarefa e sua criação aparentemente foi apenas para ser uma Tarefa que possui prioridade, ou seja, depende de Tarefa.

4. c
Uma instancia de Tarefa está recebendo TarefaPrioritaria, o que quer dizer que essa Tarefa terá um atributo a mais, que é prioridade.

4. d
Polimorfismo

5. a
Criar uma lista.

5. b
ArrayList cria uma lista, o tipo da lista é Tarefa, ou seja, nessa lista só será possível depositar dados do tipo Tarefa.

5. c
Somente TaskManager pode ter acesso a essa lista.

5. d

```
public void adicionarTarefa(Tarefa tarefa) {  
    tarefas.add(tarefa);  
}
```

6. a
0

6. b
1

6. c
2

6.d
Revisar conteúdo

6. e

Tarefa 1: Estudar Java

Tarefa 2: Fazer exercício

7. Nesse código basicamente é como se ele estivesse falando: Para cada Tarefa em tarefas(ArrayList), imprima o seu título.

Estudar Java – imprime na primeira volta com index 0 (index não é imprimido)

Fazer exercício – na segunda com index 1 ‘

Revisar conteúdo – na terceira com index 2 ‘

8. a

Listar tarefas

8. b

Opção inválida

8.c

switch case

9.

Remover tarefas

Quando a execução acha um resultado correspondente ao parâmetro do switch, para que ele pare e saia do fluxo após executar o que precisa executar, ele busca um break, se ele não encontrar, isso quer dizer que ainda existem atividades para serem executadas.

10. a

while

10. b

Para executar atividades que precisam ser executadas mais de uma vez, sejam elas com uma quantidade determinada ou indeterminada.

10. c

O sistema roda novamente esse nó do código

11. a

Pode acabar parando a execução do sistema

11. b

Tenta executar um bloco de código

11. c

Serve para impedir que o sistema pare por causa de uma exception, fazendo com que o dev trate a exception e o sistema não pare.

11. d

Para que o sistema não pare e a exception seja tratada

12.

```
public void concluirTarefa(int indice) {
```

```
Tarefa tarefa = tarefas.get(indice);  
// complete o código  
tarefa.setConcluido(true);  
}
```

Uma exception seria acionada, se existe um tratamento como try/catch poderia executar uma função para pedir um indice valido.

```
13.  
public void removerTarefa(int indice) {  
// complete o código  
Tarefa tarefa = tarefas.get(indice);  
tarefas.remove(tarefa);  
}
```

Dependencia. Verificar também se é essa atividade mesmo que o usuario deseja. Verificar se a tarefa existe.